Training Data: Acquisition, Processing, and Use

Traffic Violation Detection (AID 728)

1 Overview

The violation detector (YOLOv8n) is fine-tuned on a merged dataset of **11,479 images** assembled from five publicly available Roboflow Universe sources. The mix was selected to (i) strengthen the previously-weak `no_helmet` class, (ii) add Indian-style license-plate imagery for OCR domain match, and (iii) cover the four target classes (`helmet`, `no_helmet`, `motorcycle`, `license_plate`) without depending on a single source. The full pipeline — from download to training-ready dataset — is reproducible from the scripts under `data/` and `scripts/`.

2 Sources

2.1 Summary Table

Source	Images	Role in the merged set
Helmet & License Plate Detection (`cdio-zmfmj`, v24)	7,134	Bulk of helmet / no-helmet / license-plate annotations. Largest single contributor.
Motorcycle License Plate Detection (`motorcycle-9gyny`, v8)	2,015	Primary source of `motorcycle` class examples plus additional plates.
Traffic Violation Detection (TVD) (`traffic-violation-detection`, v11)	1,675	Indian traffic context. Only `No helmet` class is used; other classes (`Triple riding`, `Using mobile`, `Wheeling`) are dropped to avoid schema pollution.
With No Helmet (`traffic-violation`, v3)	502	Small but perfectly aligned — all four target classes present, 1:1 mapping.
Indian License Plate Detection (`license-plate-detection-khhkb`, v1)	833	Indian-style plate appearance for OCR domain match.
Total (raw)	**12,159**	
After class-filtering	**11,479**	9,758 train / 1,721 val (85/15 split, seed 42)

Table 1: Datasets used to train the violation detector.

2.2 Per-Dataset Detail

(1) Helmet & License Plate Detection (`cdio-zmfmj`, v24). The largest contributor (7,134 images). Annotations cover three classes: `LP` (license plate), `helmet`, `no helmet`. Images

are predominantly street-level traffic scenes with one or more two-wheelers. The dataset uses standard YOLOv8 axis-aligned bounding boxes. Splits within the source are train/valid/test; the merge script consumes all three.

(2) Motorcycle License Plate Detection (`motorcycle-9gyny`, v8). Provides 2,015 images annotated with two classes: `motorcycle` and `new license plate`. The primary value of this source is its dense coverage of the `motorcycle` class, which the larger dataset (`cdio-zmfmj`) does not label.

(3) Traffic Violation Detection / TVD (`traffic-violation-detection`, v11). 1,675 images of Indian traffic scenes annotated with four violation classes: `No helmet`, `Triple riding`, `Using mobile`, `Wheeling`. Only `No helmet` maps to a class in our schema; the other three are violation-event labels that do not correspond to bounding boxes of physical objects (helmet/-motorcycle/plate). The merge script therefore drops them rather than misclassifying them.

(4) With No Helmet (`traffic-violation`, v3). 502 images with four classes: `Helmet`, `motorcycle`, `nohelmet`, `platenumber`. Although small, this is the only source whose class schema aligns 1:1 with our targets, making it the most "schema-clean" contribution.

(5) Indian License Plate Detection (`license-plate-detection-khhkb`, v1). 833 images of Indian-style license plates. Used specifically to bias the `license_plate` detector toward the format and appearance of plates that the EasyOCR backend will encounter at inference time — a domain-match intervention rather than a class-balance one.

3 Acquisition Pipeline

All five datasets are downloaded from Roboflow Universe using the Roboflow Python SDK. The download targets the YOLOv8 export format (axis-aligned bounding boxes), which Ultralytics consumes directly.

```
from roboflow import Roboflow
rf = Roboflow(api_key=API_KEY)
project = rf.workspace(WORKSPACE).project(PROJECT)
project.version(VERSION).download("yolov8", location="data/tmp/<name>")
```

Each download produces the standard Roboflow YOLOv8 layout:

```
data/tmp/<name>/
  data.yaml             <- class names, paths
  train/images/*.jpg
  train/labels/*.txt    <- "<class_id> <x_c> <y_c> <w> <h>" per line,
      normalized
  valid/images/*.jpg
  valid/labels/*.txt
  test/images/*.jpg
  test/labels/*.txt
```

The five resulting directories (`helmet_lp`, `motorcycle_lp`, `tvd`, `with_no_helmet`, `indian_plates`) are the input to the merge step.

4 Class Mapping

Every source uses its own class ordering and naming. The merge script (`data/merge_new_datasets.py`) maps each source class ID to the unified four-class schema below. Source classes that are not in the target schema are silently dropped on a per-annotation basis.

Source	Source class	→	Target class (ID)
Helmet & License Plate	`LP`	→	`license_plate` (3)
	`helmet`	→	`helmet` (0)
	`no helmet`	→	`no_helmet` (1)
Motorcycle License Plate	`motorcycle`	→	`motorcycle` (2)
	`new license plate`	→	`license_plate` (3)
TVD	`No helmet`	→	`no_helmet` (1)
	`Triple riding`	→	*dropped*
	`Using mobile`	→	*dropped*
	`Wheeling`	→	*dropped*
With No Helmet	`Helmet`	→	`helmet` (0)
	`motorcycle`	→	`motorcycle` (2)
	`nohelmet`	→	`no_helmet` (1)
	`platenumber`	→	`license_plate` (3)
Indian License Plate	`license_plate`	→	`license_plate` (3)

Why drop TVD's other classes? `Triple riding` is annotated as a bounding box over the violation *event* (multiple riders on one bike), not over a specific object class in our schema. Mapping it to `motorcycle` would teach the detector to fire on the entire rider-cluster region rather than the bike body, hurting both `motorcycle` precision and downstream rider-counting via the pose detector. `Using mobile` and `Wheeling` are similarly out of scope. The merge script therefore drops these annotations on a per-line basis in the YOLO `.txt` label files.

5 Merge Pipeline

The merge script `data/merge_new_datasets.py` produces a single training-ready dataset at `data/merged_dataset/`. The pipeline executes the following steps:

1. **Source enumeration.** For each of the five source roots, `collect_pairs()` walks `train/`, `valid/`, and `test/` subdirectories, collecting every `(image_path, label_path)` pair whose extension is in `{.jpg, .jpeg, .png, .bmp, .webp}`. Images without a corresponding label file are skipped.

2. **Combination.** All pairs from all sources are concatenated into one list, tagged with their source-specific class mapping and source name.

3. **Deterministic shuffle.** The list is shuffled with `random.Random(42)` for reproducibility, then split into validation (15%) and training (85%) by simple prefix indexing.

4. **Label remap and write.** For each pair, `remap_label_file()` reads the source YOLO `.txt`, drops lines whose class ID is not in the source's class map, and rewrites the remaining lines with the remapped class ID. If *no* lines survive the filter, both the image and the label are skipped (avoids empty-label samples that confuse YOLO).

5. **Image copy.** The image is copied (`shutil.copy2`, preserves mtime) into the merged dataset with a stable, collision-free filename: `<source_name>_<idx:06d>.<ext>`.

6. **`data.yaml` emission.** A new `data.yaml` is written declaring the four target class names and the train/val image directories.

6 Resulting Layout

```
data/merged_dataset/
  data.yaml                          <- nc: 4, names: [helmet, no_helmet,
      motorcycle, license_plate]
  images/
    train/   helmet_lp_000000.jpg  (9,758 files)
             motorcycle_lp_000123.jpg
             tvd_000041.jpg
             ...
    val/     ...                    (1,721 files)
  labels/
    train/   helmet_lp_000000.txt
             ...
    val/     ...
```

The image filename prefix preserves provenance, so per-source diagnostics remain possible without maintaining a separate manifest file.

7 Data Augmentation at Training Time

Augmentations are applied by Ultralytics at training time and not baked into the on-disk dataset. The training script (`scripts/train_violation_detector.py`) overrides a few defaults for the macOS / MPS environment:

- `cache="disk"` — preprocessed images are cached to disk between epochs, avoiding repeated decode while staying under RAM limits (a full-RAM cache of 11k images at 640×640×3 is roughly 13 GB).
- `workers=4` — macOS multiprocessing in PyTorch DataLoaders is less stable with the default of 8; four workers gives the best throughput / stability tradeoff on M-series chips.
- `amp=True` — automatic mixed precision on MPS gives a ∼20% speedup with no quality regression.
- `cos_lr=True` — cosine learning-rate decay converges to a better minimum than linear decay when total epoch count is small.
- `close_mosaic=3` — mosaic augmentation is disabled in the final three epochs. Mosaic mixes four images into one to expose the detector to multi-context scenes, but training only on mosaicked images at the end of a short schedule biases the detector away from natural, un-mosaicked test-time images.

All other augmentations follow Ultralytics defaults: random HSV jitter, horizontal flip, translation, scale, and mosaic (epochs 1–7 of 10).

8 Training-Run Statistics

8.1 Validation Performance (10 Epochs, MPS)

Class	P	R	$\mathbf{mAP}_{50}$	$\mathbf{mAP}_{50–95}$
all	0.927	0.933	**0.970**	0.707
helmet	0.926	0.948	0.970	0.721
no_helmet	0.909	0.895	0.954	0.576
motorcycle	0.907	0.931	0.969	0.711
license_plate	0.967	0.957	0.985	0.818

Inference speed: 17.3 ms per image (single-image batch on M5 MPS, ∼58 FPS).

8.2 Observed Class Distribution

The five sources contribute unequally per class. `license_plate` is the most frequent (provided by four of five sources); `no_helmet` and `helmet` are dominated by the `cdio-zmfmj` source plus TVD's `No helmet` contribution. The `motorcycle` class is the most imbalanced, drawing primarily from the smaller `motorcycle-9gyny` and `with_no_helmet` sources. Despite this skew, all four classes reach $\text{mAP}_{50} \geq 0.954$ at convergence, suggesting the imbalance is not the binding constraint at the current model capacity.

9 Quality Considerations and Limitations

Label-style heterogeneity. The five sources were annotated by different teams using different conventions. Some draw helmet boxes tight around the helmet only; others include the full head; a few include shoulders. No relabeling was performed; YOLO is expected to absorb the inconsistency.

Domain shift. The original course-project pipeline used Indian-traffic datasets that are no longer available. The current sources are heterogeneous: TVD (3) and Indian License Plate (5) are Indian-context; sources 1, 2, and 4 are more international. This mix likely under-represents some Indian-specific patterns (e.g., night-time scenes with mixed two-wheeler types, helmet colors/styles, plate fonts) and may bias the detector toward generic appearances.

Dropped TVD classes. Discarding `Triple riding`, `Using mobile`, and `Wheeling` sacrifices roughly half of TVD's annotations. This was a deliberate trade — mapping them to any target class would inject noise — but it does mean those 1,675 images contribute fewer training signals than their image count suggests.

Test-set leakage. Each source's internal `test/` split is folded into the merged training pool. Source-level test sets are therefore not preserved. Held-out evaluation is performed only on the 15% merged validation split. For a more rigorous evaluation, a future iteration could maintain per-source held-out test splits.

Schema-level losses. Some source annotations would be useful with a richer schema (e.g., `Triple riding` as auxiliary supervision for rider-counting). The current four-class schema is dictated by the spec; an auxiliary multi-task head would be required to make use of these labels.

10 Reproducibility

Given a fresh checkout, the merged dataset can be rebuilt in three commands:

```
# 1. Download all five sources from Roboflow into data/tmp/
.venv/bin/python -c "
from roboflow import Roboflow
rf = Roboflow(api_key='YOUR_KEY')
for ws, proj, ver, loc in [
    ('cdio-zmfmj', 'helmet-lincense-plate-detection-gevlq', 24, 'data/tmp/
        helmet_lp'),
    ('motorcycle-9gyny', 'motorcycle-license-plate-detection', 8, 'data/tmp/
        motorcycle_lp'),
    ('traffic-violation-detection', 'tvd-kp9qw', 11, 'data/tmp/tvd'),
    ('traffic-violation', 'with-no-helmet', 3, 'data/tmp/with_no_helmet'),
    ('license-plate-detection-khhkb', 'indian-license-plate-detection-6tmbr',
        1, 'data/tmp/indian_plates'),
]:
    rf.workspace(ws).project(proj).version(ver).download('yolov8', location=
        loc)
"
```

```
# 2. Run the merge script
.venv/bin/python data/merge_new_datasets.py

# 3. Train
.venv/bin/python scripts/train_violation_detector.py \
    --data data/merged_dataset/data.yaml --model yolov8n.pt \
    --epochs 10 --imgsz 640 --batch 16 --device mps
```

The fixed random seed (42) and the deterministic enumeration order in `collect_pairs()` guarantee a bit-for-bit identical merged dataset across machines.

11 Licenses

- *Helmet & License Plate Detection* — CC BY 4.0
- *Motorcycle License Plate Detection* — Public Domain
- *Traffic Violation Detection (TVD)* — Public Domain
- *With No Helmet* — CC BY 4.0
- *Indian License Plate Detection* — CC BY 4.0